

Clase și obiecte. Instanțe

- **Clasa:** Un șablon sau plan care definește proprietăți (câmpuri) și comportament (metode).
- **Obiect:** O instanță a unei clase.
- **Instanță:** Un obiect concret creat din clasă folosind `new`.

```
Dog myDog = new Dog(); // myDog este o instanță a clasei D
```

Modificatori de acces

- `default` (**fără modificador**): Accesibil doar în cadrul pachetului.

Public

- **Public** face ca elementul (metodă, variabilă, clasă) să fie accesibil din orice altă clasă, indiferent de pachetul în care se află.
- De exemplu, o metodă **public** poate fi apelată de orice altă clasă din orice pachet.

Private

- **Private** restricționează accesul la un element astfel încât acesta poate fi accesat doar în cadrul aceleiași clase în care este definit.
- Acesta este cel mai restricționat nivel de acces și protejează datele de accesul extern.

Protected

- **Protected** permite accesul la un element în cadrul aceleiași clase, în cadrul claselor din același pachet și în clasele care moștenesc clasa respectivă (adică subclasele).
- Dacă o clasă derivată se află într-un alt pachet, tot va putea accesa elementele protected ale clasei părinte.

Static

- **Static** face ca elementul să fie legat de clasa în sine și nu de o instanță a clasei.
- Aceasta înseamnă că variabilele și metodele statice sunt accesibile fără a crea o instanță a clasei. Ele sunt partajate de toate instanțele clasei.
- De obicei se folosesc pentru valori sau funcții care nu depind de starea unui obiect, ci sunt comune pentru toate instanțele clasei.

Final

- **Final** este folosit pentru a declara elemente care nu pot fi modificate după ce au fost inițializate:
 - **final class**: o clasă nu poate fi moștenită.
 - **final method**: o metodă nu poate fi suprascrisă în subclase.
 - **final variable**: o variabilă nu poate fi reatribuită după ce a fost inițializată.

```
public final class Example {  
    // nu poate fi mostenita  
    public final void display() {  
        System.out.println("This is a final method.");  
    }  
}  
  
public class SubExample extends Example {  
    // nu poate suprascrie display(), pentru ca este final  
}
```

Principii OOP

1. Încapsulare (Encapsulation)

- **Definiție**: Ascunderea detaliilor interne ale unui obiect și expunerea doar a ceea ce este necesar.
- **Cum se realizează**:
 - Folosirea modificatorilor de acces (`private`, `public`, `protected`).
 - Furnizarea de metode `getter` și `setter` pentru acces controlat la câmpuri.
- **Exemplu**:

```
class Person {  
    private String name; // Câmp privat  
    public String getName() { // Getter  
        return name;  
    }  
    public void setName(String name) { // Setter  
        this.name = name;  
    }  
}
```

2. Moștenire (Inheritance)

- **Definiție**: Capacitatea unei clase de a prelua proprietăți și metode ale unei alte clase.

- **Cum se realizează:**
 - Folosind cuvântul cheie `extends`.
 - Clasa care moștenește se numește **subclasă** (sau clasă derivată).
 - Clasa de la care se moștenește se numește **superclasă** (sau clasă de bază).
- **Exemplu:**

```
class Animal { // Superclasă
    void sound() {
        System.out.println("Animal sound");
    }
}
class Dog extends Animal { // Subclasă
    @Override
    void sound() {
        System.out.println("Bark");
    }
}
```

3. Polimorfism (Polymorphism)

- **Definiție:** Capacitatea unui obiect de a se comporta în mai multe moduri, în funcție de context.
- **Tipuri:**
 1. **Polimorfism de runtime (suprascrierea metodelor):**
 - O metodă are implementări diferite în clasele derivate.
 - Exemplu:

```
Animal myDog = new Dog();
myDog.sound(); // Va apela metoda din Dog
```

2. **Polimorfism de compile-time (supraîncărcarea metodelor):**
 - Mai multe metode cu același nume, dar cu parametri diferiți.
 - Exemplu:

```
void print(int x) { System.out.println(x); }
void print(String s) { System.out.println(s); }
```

4. Abstractizare (Abstraction)

- **Definiție:** Simplificarea complexității prin expunerea doar a caracteristicilor esențiale ale unui obiect.
- **Cum se realizează:**
 - Folosind **clase abstracte** și **interfețe**.
 - Clasele abstracte pot conține metode abstracte (fără implementare) și metode implementate.
 - Interfețele definesc un contract (metode abstracte) care trebuie implementat de clase.
- **Exemplu clasă abstractă:**

```
abstract class Animal {
    abstract void sound(); // Metodă abstractă
    void sleep() { // Metodă implementată
        System.out.println("Sleeping");
    }
}
```

- **Exemplu interfață:**

```
interface Sound {
    void makeSound(); // Metodă abstractă
}
class Dog implements Sound {
    public void makeSound() {
        System.out.println("Bark");
    }
}
```

5. Asociere (Association)

- **Definiție:** Relația între două clase independente, unde un obiect folosește alt obiect.
- **Tipuri:**
 1. **Agregare (Aggregation):** Relație "has-a" unde obiectul conținut poate exista independent.
 - Exemplu: Un departament are profesori, dar profesorii pot exista fără departament.

```
class Department {
    private List<Professor> professors;
    Department(List<Professor> professors) {
```

```
        this.professors = professors;
    }
}
```

2. **Compunere (Composition):** Relație "part-of" unde obiectul conținut nu poate exista independent.

- Exemplu: O mașină are un motor, iar motorul nu poate exista fără mașină.

```
class Engine { ... }
class Car {
    private Engine engine;
    Car() {
        this.engine = new Engine(); // Motorul este creat odată cu
        mașina
    }
}
```

6. Colectarea gunoiului (Garbage Collection)

- **Definiție:** Procesul automat de gestionare a memoriei în Java, care elimină obiectele nefolosite.
- **Cum funcționează:**
 - JVM identifică și șterge obiectele care nu mai sunt referite.
 - Dezvoltatorul nu trebuie să elibereze manual memoria.

7. Legarea dinamică (Dynamic Binding)

- **Definiție:** Metoda care trebuie apelată este determinată la runtime, în funcție de tipul obiectului.
- **Exemplu:**

```
Animal myDog = new Dog();
myDog.sound(); // Metoda sound() din Dog este apelată, nu cea din Animal
```

8. Încapsularea datelor (Data Hiding)

- **Definiție:** Ascunderea detaliilor de implementare și expunerea doar a funcționalităților necesare.
- **Exemplu:**

```
class BankAccount {
    private double balance; // Ascuns
    public double getBalance() { // Expus
        return balance;
    }
}
```

9. Reutilizarea codului (Code Reusability)

- **Definiție:** Capacitatea de a folosi același cod în mai multe locuri.
- **Cum se realizează:**
 - Prin moștenire și compoziție.
 - Exemplu:

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
}
class ScientificCalculator extends Calculator {
    // Poate folosi metoda add() din Calculator
}
```

10. Modularitate (Modularity)

- **Definiție:** Împărțirea codului în module sau componente independente.
- **Cum se realizează:**
 - Folosind clase și pachete.
 - Exemplu:

```
package com.example.math;
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}
```

Clase abstracte

- O **clasă abstractă** este o clasă care nu poate fi instanțiată direct (nu poți crea obiecte din ea).
- O clasă poate extinde (extends) o singură clasă abstractă
- Este folosită ca un șablon pentru alte clase (clase derivate).
- Poate conține:
 - **Metode abstracte:** Metode fără implementare (doar semnătura).
 - **Metode concrete:** Metode cu implementare.

```
abstract class Animal {
    abstract void sound(); // Metoda abstracta

    void sleep() { // Metoda concreta
        System.out.println("Sleeping");
    }
}

class Dog extends Animal {
    @Override
    void sound() { // Implementare metodei abstracte
        System.out.println("Bark");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        myDog.sound(); // Bark
        myDog.sleep(); // Sleeping
    }
}
```

Interfețe

O **interfață** este un tip special în Java folosit pentru a defini un contract pe care clasele trebuie să-l respecte. Interfețele pot include metode abstracte, metode default, metode statice și constante.

- **Metode abstracte:** Metodele fără corp, pe care clasele implementatoare trebuie să le definească.
- **Metode default:** Metode cu implementare care pot fi utilizate direct de clasele implementatoare sau pot fi suprascrise.
- **Metode statice:** Pot fi apelate direct de pe interfață, fără o instanță.

- **Constante:** Variabilele dintr-o interfață sunt implicit `public static final`.

```
interface Vehicle {
    int MAX_SPEED = 120; // constanta (implicit public static final)

    void start(); // metoda abstracta (implicit public)

    default void stop() { // metoda default
        System.out.println("Vehicle stopped.");
    }

    static void service() { //metoda statica
        System.out.println("Service your vehicle.");
    }
}
```

Excepții

Excepțiile în **Java** sunt mecanisme utilizate pentru a gestiona erorile sau condițiile anormale care apar în timpul execuției unui program. Ele permit separarea logicii normale de tratarea erorilor și asigură un program mai robust și mai ușor de întreținut.

Tipuri de Excepții

1. Checked Exceptions (Verificate la compilare):

- Excepții pe care compilatorul obligă dezvoltatorul să le gestioneze (`try-catch` sau `throws`).
- Exemple: `IOException`, `SQLException`
- Cod exemplu:

```
import java.io.*;

public class FileReadExample {
    public void readFile() throws IOException {
        BufferedReader reader = new BufferedReader(new
        FileReader("file.txt"));
        System.out.println(reader.readLine());
    }
}
```

2. Unchecked Exceptions (Ne-verificate la compilare):

- Apar în timpul execuției și nu sunt verificate de compilator.
- Moștenesc clasa `RuntimeException`.

- Exemple: `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException`
- Cod exemplu:

```
public class DivisionExample {
    public static void main(String[] args) {
        int result = 10 / 0; // Va arunca ArithmeticException
    }
}
```

3. Errors:

- Probleme critice legate de JVM, care nu pot fi gestionate de aplicație.
- Exemple: `OutOfMemoryError`, `StackOverflowError`

Tratarea Excepțiilor

Java oferă mai multe mecanisme pentru gestionarea excepțiilor.

1. try-catch

Blochează și gestionează excepțiile care apar într-o secțiune de cod.

```
public class TryCatchExample {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // Posibilă excepție
        } catch (ArithmeticException e) {
            System.out.println("Eroare: Împărțire la zero.");
        }
    }
}
```

2. finally

Un bloc opțional care rulează întotdeauna, indiferent dacă apare o excepție sau nu.

```
public class FinallyExample {
    public static void main(String[] args) {
        try {
            int[] arr = new int[2];
            System.out.println(arr[5]); // ArrayIndexOutOfBoundsException
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Index invalid.");
        }
    }
}
```

```

        } finally {
            System.out.println("Acest bloc este întotdeauna executat.");
        }
    }
}

```

3. throw

Utilizat pentru a arunca explicit o excepție.

```

public class ThrowExample {
    public static void validateAge(int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Vârsta minimă este 18 ani.");
        }
    }

    public static void main(String[] args) {
        validateAge(16); // Va arunca IllegalArgumentException
    }
}

```

4. throws

Declară excepțiile pe care o metodă le poate arunca.

```

import java.io.*;

public class ThrowsExample {
    public void readFile() throws IOException {
        BufferedReader reader = new BufferedReader(new
        FileReader("file.txt"));
        System.out.println(reader.readLine());
    }
}

```

Crearea Excepțiilor Personalizate

Poți defini propriile excepții prin extinderea clasei `Exception` sau `RuntimeException`.

- Checked Exception Personalizată:

```

class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class CustomCheckedExample {
    public static void validateAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Vârsta minimă este 18 ani.");
        }
    }

    public static void main(String[] args) {
        try {
            validateAge(16);
        } catch (InvalidAgeException e) {
            System.out.println("Eroare: " + e.getMessage());
        }
    }
}

```

- Unchecked Exception Personalizată:

```

class InvalidAgeRuntimeException extends RuntimeException {
    public InvalidAgeRuntimeException(String message) {
        super(message);
    }
}

public class CustomUncheckedExample {
    public static void validateAge(int age) {
        if (age < 18) {
            throw new InvalidAgeRuntimeException("Vârsta minimă este 18
ani.");
        }
    }

    public static void main(String[] args) {
        validateAge(16); // Va arunca InvalidAgeRuntimeException
    }
}

```

Colecții

Colecțiile sunt structuri de date care gestionează grupuri de obiecte. Se găsesc în pachetul `java.util`.

1. List

- Colecție ordonată care permite elemente duplicate.
- Implementări populare: `ArrayList`, `LinkedList`.

```
import java.util.*;

public class ListExample {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("apple");
        list.add("banana");
        list.add("apple");
        System.out.println(list); // [apple, banana, apple]
    }
}
```

2. Set

- Colecție care NU permite duplicate (Mulțime)
- Implementări populare: `HashSet` (neordonat), `TreeSet` (ordonat).

```
import java.util.*;

public class SetExample {
    public static void main(String[] args) {
        Set<String> set = new HashSet<>();
        set.add("apple");
        set.add("banana");
        set.add("apple");
        System.out.println(set); // [banana, apple]
    }
}
```

3. Map

- Stochează perechi cheie-valoare (Dicționar)
- Implementări populare: `HashMap` (neordonat), `TreeMap` (ordonat).

```
import java.util.*;

public class MapExample {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("apple", 10);
        map.put("banana", 5);
        System.out.println(map); // {apple=10, banana=5}
    }
}
```

4. Iteratori

- Utilizați pentru a traversa colecțiile.

```
import java.util.*;

public class IteratorExample {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("apple", "banana", "cherry");
        Iterator<String> iterator = list.iterator();
        while (iterator.hasNext()) {
            System.out.println(iterator.next());
        }
    }
}
```

5. Comparatori (Comparator și Comparable)

Comparable: Folosit pentru ordonare naturală.

- Implementare în clasă.

```
import java.util.*;

class Fruit implements Comparable<Fruit> {
    String name;
    int quantity;
```

```

    Fruit(String name, int quantity) {
        this.name = name;
        this.quantity = quantity;
    }

    @Override
    public int compareTo(Fruit other) {
        return this.name.compareTo(other.name);
    }
}

public class ComparableExample {
    public static void main(String[] args) {
        List<Fruit> fruits = Arrays.asList(new Fruit("apple", 10), new
Fruit("banana", 5));
        Collections.sort(fruits);
        for (Fruit f : fruits) {
            System.out.println(f.name);
        }
    }
}

```

Comparator: Folosit pentru ordonare personalizată.

- Implementare în afara clasei.

```

import java.util.*;

class Fruit {
    String name;
    int quantity;

    Fruit(String name, int quantity) {
        this.name = name;
        this.quantity = quantity;
    }
}

class QuantityComparator implements Comparator<Fruit> {
    @Override
    public int compare(Fruit f1, Fruit f2) {
        return Integer.compare(f1.quantity, f2.quantity);
    }
}

```

```
public class ComparatorExample {  
    public static void main(String[] args) {  
        List<Fruit> fruits = Arrays.asList(new Fruit("apple", 10), new  
Fruit("banana", 5));  
        Collections.sort(fruits, new QuantityComparator());  
        for (Fruit f : fruits) {  
            System.out.println(f.name + ": " + f.quantity);  
        }  
    }  
}
```

Streams

Funcții lambda

Interfețe funcționale

Fișiere I/O. Serializare și deserializare

Concurență

- Fire de execuție (Thread)
- Sincronizare

Generic Types

Anotații

Design Patterns